

ANTI_BLUFF

Anti-Bluff Discipline (CONST-035) — Runbook

This document is the runbook for working with the anti-bluff gates in this repository. The rule itself lives in `CONSTITUTION.md` (CONST-035).

What "bluff" means here

A test is **bluff** if it can pass without exercising the user-visible behavior it claims to verify. In this Go submodule, "user-visible" means an operator invoking the CLI/binary can observe the result — the anchor signal is a CLI invocation against a real target producing an observable artifact (file on disk, exit code, log line) that the test asserts on.

Three gates

1. **Static scanner** (`scripts/anti-bluff/bluff-scanner.sh`) — pattern matcher that flags forbidden constructs. Runs on every commit (pre-commit hook) and in `make qa-all`.
2. **Mutation testing** (`go-mutesting`) — kills generated mutants. Threshold: 90% on changed code, 80% project-wide ratchet. Runs in `make qa-all` (slow; not in pre-commit).
3. **Behavior-anchor manifest** (`docs/behavior-anchors.md`) — every user-facing capability has at least one anchor test that proves it works end-to-end.

"I got a bluff hit, what now?"

The scanner output names the file, line, BLUFF-G-NNN ID, and a one-line context. Look up the BLUFF-G-NNN in the table below:

ID	Pattern	Fix
BLUFF-G-001	<code>t.Skip()</code> without exempt comment	Add <code>// SKIP-OK: #<ticket></code> on the line above (or <code>// ANTI-BLUFF-EXEMPT: <ticket></code> synonym), or remove the skip and fix the underlying issue.
BLUFF-G-002	if <code>testing.Short()</code> early return without long-path coverage elsewhere	Add a sub-test that runs the long path; or add the exempt comment if short is correct here.
BLUFF-G-003	Test body has only <code>t.Log</code> , no assertions	Add <code>t.Fatal/t.Error/testify</code> assertions.
BLUFF-G-004	gomock mocking a type from the same package	Stop mocking the SUT; use the real type or move the mocked type to a different package.
BLUFF-G-005	<code>t.Run("", func)</code> empty subtest body	Fill the subtest body with assertions or remove.
BLUFF-G-006	Empty test function body	Fill or delete.
BLUFF-G-007	<code>assert.True(t, true)</code> / <code>assert.NotNil(t, x)</code> as sole assertion	Add a real assertion that exercises the SUT's behavior.

Skip-exempt marker convention

The repository convention is `// SKIP-OK: #<ticket>` (the existing project-wide marker). The scanner also recognizes the synonym `// ANTI-BLUFF-EXEMPT: <ticket>` and, for mutation-equivalent exemptions, `// ANTI-BLUFF-EXEMPT: TRIVIAL-CORRECT – <reason>`. Place the marker on the line **above** the offending statement.

"Mutation gate failed on my change"

go-mutesting printed mutants that survived (the test suite did not detect them). Each surviving mutant is a place where the SUT's behavior could change without any test noticing. Either:

- Add a test that would notice (preferred), or
- Add an in-line `// ANTI-BLUFF-EXEMPT: TRIVIAL-CORRECT – <reason>` if the mutant is genuinely equivalent (the mutated code is semantically identical to the original — extremely rare).

The challenge enforces 90% kill rate on changed files. Equivalent mutants count toward the 10% slack; you should not need exemptions unless your change happens to hit a mathematical identity.

"Anchor manifest check failed"

`anchor_manifest_challenge.sh` validates `docs/behavior-anchors.md`. Most failures are: the `anchor_test_path` you wrote does not resolve to an existing test method. Re-check the path; the format is `<relative path>.go::TestName` for Go.

Reducing the baseline

The baseline file `challenges/baselines/bluff-baseline.txt` is expected to shrink during sub-project 4. Removing a line is a **ratchet improvement**: do it in the same commit that fixes the underlying bluff. The scanner exits with code 2 if it sees a baselined hit that is no longer present — this is the signal that the baseline file is stale.

Verification commands

Run all three before declaring work done:

```
bash scripts/anti-bluff/bluff-scanner.sh --mode all
bash challenges/scripts/anchor_manifest_challenge.sh
bash challenges/scripts/mutation_ratchet_challenge.sh
```

All three must pass.